StreamVerse Media Engineering Hackathon

STRINGS, ARRAYS & LINKED LISTS · PROBLEM STATEMENTS

Developed by **TalenciaGlobal**

StreamVerse is a media and entertainment streaming platform. Across five levels you will build the engineering modules behind it: caption analytics, viewership trends, playlist management, script search and repeated-scene detection.

Each level states the inputs, the required outputs and the formulas you may rely on. The levels build on one another and grow harder — from basic string handling to the KMP and Rabin-Karp algorithms. No starter code is provided; design and write each solution yourself.

Levels

Level 1 — Caption Analyzer

★ Beginner

Level 2 — Viewership Trends

★★ Beginner+

Level 3 — Playlist Manager

★★★ Intermediate

Level 4 — Script Search

★★★★ Advanced

Level 5 — Repeated Scene Detector

★★★★★ Expert

Level 1 — Caption Analyzer

☆ BEGINNER STRINGS

Scenario

StreamVerse processes millions of subtitle captions. Given one caption line, report basic string metrics and two transformations used by the captioning pipeline.

Concepts Covered

- Splitting a string into words
- Counting words and characters
- Reversing word order
- Title-casing each word

Problem Statement

Given: A single caption line of space-separated words

Required output:

- The word count
- The character count (letters only, spaces excluded)
- The caption with the word order reversed
- The caption in Title Case

Constraints: Words are separated by single spaces. Each word has at least one character.

Input: One line of text. **Output:** Four lines: Words, Characters, Reversed, Title Case.

Formula Guidance

characters = sum of word lengths (spaces excluded)

reversed = the words in reverse order

title case = the first letter of each word upper-cased

Sample Input

```
the dark knight rises again
```

Sample Output

```
Words: 5
Characters: 23
Reversed: again rises knight dark the
Title Case: The Dark Knight Rises Again
```

⚠️ Common Pitfall

Count characters across the words only (excluding the separating spaces), and capitalise just the first letter of each word.

💡 Thinking Skill

Most string work reduces to splitting, scanning and rebuilding — master those three and the rest follows.

Level 2 — Viewership Trends

BEGINNER+ ARRAYS

Scenario

Analyse a show's daily viewership over a window of days: totals, averages, the peak day, and the longest run of consecutive days where viewership kept rising (its momentum streak).

Concepts Covered

- Summing and averaging an array
- Tracking a running maximum
- Detecting the longest strictly-increasing run
- Single-pass aggregation

Constraints

- 1 <= n <= 10^5
- View counts are non-negative integers
- A rising streak counts consecutive days each strictly greater than the day before

Problem Statement

Given: A day count n, then n daily view counts

Required output:

- Total views
- Average views (two decimals)
- Peak single-day views
- The length of the longest strictly-rising streak (in days)

Input: Line 1: n. Line 2: n daily view counts.

Output: Four lines: Total Views, Average Views, Peak Day Views, Longest Rising Streak.

Sample Input

```
8
10 20 15 18 25 30 12 14
```

Sample Output

```
Total Views: 144
Average Views: 18.00
Peak Day Views: 30
Longest Rising Streak: 4 days
```

Formula Guidance

average = total / n

rising streak = the longest run with views[i] > views[i-1]

⚠️ Common Pitfall

A streak resets the moment views stop strictly increasing; equal consecutive days break the run.

💡 Thinking Skill

Several array statistics — sum, max and a longest-run — can all be gathered in a single pass.

Level 3 — Playlist Manager

☆ INTERMEDIATE | LINKED LISTS

Scenario

A user's playlist is a singly linked list of track IDs. Build it, then perform the four operations every playlist UI needs: list it, reverse it, find the middle track, and find the k-th track from the end.

Concepts Covered

- Building a singly linked list
- Traversal
- In-place reversal by pointer rewiring
- Slow/fast pointers for the middle
- Two-pointer k-th-from-end

Problem Statement

Given: A track count n, then n track IDs, then an integer k

Required output:

- The playlist in order
- The reversed playlist
- The middle track
- The k-th track from the end

Constraints: 1 <= n <= 10^5 | 1 <= k <= n | For an even-length list the middle is the second of the two central nodes.

Input: Line 1: n. Line 2: n track IDs. Line 3: k.

Output: Four lines: Playlist, Reversed, Middle, Kth From End.

Formula Guidance

middle: advance slow by 1 and fast by 2 until fast falls off the end

k-th from end: advance a lead pointer k steps, then move lead and trail together

Sample Input

```
5
10 20 30 40 50
2
```

Sample Output

```
Playlist: 10 20 30 40 50
Reversed: 50 40 30 20 10
Middle: 30
Kth From End: 40
```

⚠️ Common Pitfall

Capture the middle and the k-th-from-end **before** reversing, since reversal rewires the very pointers those traversals rely on.

💡 Thinking Skill

Two coordinated pointers — slow/fast or lead/trail — solve a surprising number of linked-list problems in a single pass.

Level 4 — Script Search

☆ ADVANCED | KMP PATTERN MATCHING

Scenario

Search a movie script for a phrase. Use the **Knuth-Morris-Pratt algorithm** to count every (overlapping) occurrence, report the first position, and expose the prefix table that makes the search linear.

Concepts Covered

- The longest-proper-prefix-suffix (LPS) table
- Linear-time pattern matching
- Counting overlapping matches
- Never backtracking in the text

Constraints

- Text up to 10^5 characters
- Occurrences may overlap and are all counted
- Positions are 1-based

Problem Statement

Given: A text line and a pattern line

Required output:

- The number of overlapping occurrences
- The 1-based position of the first occurrence (or -1)
- The LPS table of the pattern

Input: Line 1: the text. Line 2: the pattern.

Output: Three lines: Occurrences, First Position, LPS.

Sample Input

```
abababab
abab
```

Sample Output

```
Occurrences: 3
First Position: 1
LPS: 0 0 1 2
```

Formula Guidance

LPS[i] = length of the longest proper prefix of pattern[0..i] that is also a suffix

On a full match, reset the match length via LPS to allow overlaps.

⚠️ Common Pitfall

Reset the match length with LPS after a full match (not to 0), or overlapping occurrences like `abab` inside `abababab` are missed.

💡 Thinking Skill

The LPS table is the heart of KMP: it tells you exactly how far to fall back without ever re-reading the text.

Level 5 — Repeated Scene Detector

☆ EXPERT | RABIN-KARP ROLLING HASH

Scenario

Detect repeated scenes in a script by finding which fixed-length segments occur more than once. A **rolling hash (Rabin-Karp)** compares many substrings cheaply; verify candidates to guard against hash collisions.

Concepts Covered

- Polynomial rolling hash
- Sliding the hash in O(1) per step
- Bucketing by hash with verification
- Counting distinct and repeated segments

Problem Statement

Given: A script string s and a window length L

Required output:

- The number of distinct length-L substrings
- How many distinct substrings repeat (occur at least twice)
- The earliest-starting repeated substring (or *none*)

Constraints: 1 <= L <= length of s | Confirm a hash match by comparing characters | Report *none* if no substring repeats.

Input: Line 1: the script string s. Line 2: the window length L.

Output: Three lines: Distinct Substrings, Repeated Substrings, First Repeat.

Formula Guidance

Rolling hash:

`h(i) = (h(i-1) - s[i-1]*BASE^(L-1)) * BASE + s[i+L-1]` (mod a large prime)

Bucket windows by hash, then verify equality before counting.

Sample Input

```
abcabcabc
3
```

Sample Output

```
Distinct Substrings: 3
Repeated Substrings: 3
First Repeat: abc
```

⚠️ Common Pitfall

Hashes can collide, so confirm a real character match before counting two windows as equal; the reported counts stay exact regardless of the hash.

💡 Thinking Skill

A rolling hash turns repeated-substring comparison from O(L) into O(1) per step — but always verify a hit, because hashes can collide.